

```

3 {
4     int n;
5     scanf("%d",&n);
6     int arr[n];
7     for(int i=0;i<n;i++)
8     scanf("%d",&arr[i]);
9     int max=arr[0];
10    for(int i=1;i<n;i++)
11    {
12        if(arr[i]>max)
13            max=arr[i];
14    }
15    max++;
16    int min=0;
17    for(int a=0;a<n;a++)
18    {
19        for(int b=0;b<n;b++)
20        {
21            if(arr[b]<arr[min])
22                min=b;
23        }
24        printf("%d ",min);
25        arr[min]=max;
26    }
27
28 }

```

	Input	Expected	Got	
✓	5 4 5 3 7 1	4 2 0 1 3	4 2 0 1 3	✓

Passed all tests! ✓

OUTPUT :

Output consists of a single line of integers

CONSTRAINTS:

$1 \leq m \leq 106$

$0 \leq A[i] \leq 106$

NOTE: The indexing of the array starts with 0.

SAMPLE INPUT

5
4 5 3 7 1

SAMPLE OUTPUT

4 2 0 1 3

Answer: (penalty regime: 0 %)

1	#include<stdio.h>
2	int main()

	Input	Expected	Got	
✓	5 1 3 1 4 3	2	2	✓

Passed all tests! ✓

You are given an array **A** of non-negative integers of size **m**. Your task is to sort the array in non-decreasing order and print out the original indices of the new sorted array.

Example:

A={4,5,3,7,1}

After sorting the new array becomes A={1,3,4,5,7}.

The required output should be "4 2 0 1 3"

INPUT :

The first line of input consists of the size of the array

The next line consists of the array of size m

2

Explanation

The 2 pair of indices are **(1, 3)** and **(2,5)**.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,count=0;
5     scanf("%d",&n);
6     int arr[n];
7     for(int i=0;i<n;i++)
8         scanf("%d",&arr[i]);
9     for(int i=0;i<n-1;i++)
10    {
11        for(int j=i+1;j<n;j++)
12        {
13            if((arr[i]^arr[j])==0)
14                count++;
15        }
16    }
17    printf("%d",count);
18
19 }
```

You are given an array of n integer numbers $a_1, a_2, \dots, a_n$. Calculate the number of pair of indices (i, j) such that $1 \leq i < j \leq n$ and $a_i \text{ xor } a_j = 0$.

Input format

- First line: n denoting the number of array elements
- Second line: n space separated integers $a_1, a_2, \dots, a_n$.

Output format

Output the required number of pairs.

Constraints

$$1 \leq n \leq 10^6$$

$$1 \leq a_i \leq 10^9$$

SAMPLE INPUT

5

1 3 1 4 3

SAMPLE OUTPUT

```

22     vac[min1]=pat[j];
23     pat[j]=temp;
24
25     temp=pat[min2];
26     pat[min2]=pat[j];
27     pat[j]=temp;
28 }
29 for(int i=0;i<n;i++)
30 {
31     if(vac[i]<=pat[i])
32     {
33         flag=0;
34         break;
35     }
36 }
37 if(flag==1)
38     printf("Yes");
39 else
40     printf("No");
41 }

```

	Input	Expected	Got	
✓	5 123 146 454 542 456 100 328 248 689 200	No	No	✓

Passed all tests! ✓

123 146 454 542 456

100 328 248 689 200

SAMPLE OUTPUT

No

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,min1,min2,temp,flag=1;
5     scanf("%d",&n);
6     int vac[n],pat[n];
7     for(int i=0;i<n;i++)
8         scanf("%d",&vac[i]);
9     for(int i=0;i<n;i++)
10         scanf("%d",&pat[i]);
11     for(int j=0;j<n-1;j++)
12     {
13         min1=j,min2=j;
14         for(int k=j;k<n;k++)
15         {
16             if(vac[k]<vac[min1])
17                 min1=k;
18             if(pat[k]<pat[min2])
19                 min2=k;
20         }
21         temp=vac[min1];
```

A new deadly virus has infected large population of a planet. A brilliant scientist has discovered a new strain of virus which can cure this disease. Vaccine produced from this virus has various strength depending on midichlorians count. A person is cured only if midichlorians count in vaccine batch is more than midichlorians count of person. A doctor receives a new set of report which contains midichlorians count of each infected patient, Practo stores all vaccine doctor has and their midichlorians count. You need to determine if doctor can save all patients with the vaccines he has. The number of vaccines and patients are equal.

Input Format

First line contains the number of vaccines - N. Second line contains N integers, which are strength of vaccines. Third line contains N integers, which are midichlorians count of patients.

Output Format

Print a single line containing '**Yes**' or '**No**'.

Input Constraint

$$1 < N < 10$$

Strength of vaccines and midichlorians count of patients fit in integer.

SAMPLE INPUT

5

```

13 scanf("%d",&arr[i]);
14 for(int j=0;j<n;j++)
15 {
16 min=j;
17 for(int k=j;k<n;k++)
18 {
19 if(arr[k]<arr[min])
20 min=k;
21 }
22 temp=arr[min];
23 arr[min]=arr[j];
24 arr[j]=temp;
25 }
26 int maxsum=0,minsum=0;
27 for(int a=0;a<d;a++)
28 minsum+=arr[a];
29 for(int b=n-1;b>m-1;b--)
30 maxsum+=arr[b];
31 printf("%d\n",maxsum-minsum);
32 }
33 }
34

```

	Input	Expected	Got	
✓	1 5 1 1 2 3 4 5	4	4	✓

Passed all tests! ✓

5 1

1 2 3 4 5

SAMPLE OUTPUT

4

Explanation

M is 1 and N is 5 so you have to calculate maximum and minimum sum using $(5-1 =)$ 4 elements.

Maximum sum using the 4 elements would be $(2+3+4+5=)$ 14.

Minimum sum using the 4 elements would be $(1+2+3+4=)$ 10.

Difference will be $14-10=$ 4.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int t;
5     scanf("%d",&t);
6     while(t--)
7     {
8         int n,m,d,min,temp;
9         scanf("%d %d",&n,&m);
10        d=n-m;
11        int arr[n];
12        for(int i=0;i<n;i++)
```

Coders here is a simple task for you, you have given an array of size ***N*** and an integer ***M***.

Your task is to calculate the ***difference between maximum sum and minimum sum of N-M*** elements of the given array.

Constraints:

$$1 \leq t \leq 10$$

$$1 \leq n \leq 1000$$

$$1 \leq a[i] \leq 1000$$

Input:

First line contains an integer ***T*** denoting the number of testcases.

First line of every testcase contains two integer ***N*** and ***M***.

Next line contains ***N*** space separated integers denoting the elements of array

Output:

For every test case print your answer in new line

SAMPLE INPUT

1

```

9      int arr[n];
10     for(int j=0;j<n;j++)
11         scanf("%d",&arr[j]);
12     m=n/2;
13     if(arr[m]==0){
14         for(m=0;arr[m]==0 && m<n;m++);
15     }
16     for(int j=0;j<=m;j++)
17         Is=Is+arr[j];
18     for(int j=m;j<n;j++)
19         rs=rs+arr[j];
20     printf("%s\n",(Is==rs)?"YES":"NO");
21 }
22 return 0;
23 }

```

	Input	Expected	Got	
✓	3	YES	YES	✓
	5	YES	YES	
	1 1 4 1 1	YES	YES	
	4			
	2 0 0 0			
	4			
	0 0 2 0			
✓	2	NO	NO	✓
	3	YES	YES	
	1 2 3			
	4			
	1 2 3 3			

Passed all tests! ✓

4
2 0 0 0
4
0 0 2 0

Sample Output 1

YES
YES
YES

Explanation 1

In the first test case, **arr[2] = 4** is between two subarrays summing to **2**.
In the second case, **arr[0] = 2** is between two subarrays summing to **0**.
In the third case, **arr[2] = 2** is between two subarrays summing to **0**.

Answer: (penalty regime: 0 %)

1	#include<stdio.h>
2	int main(){
3	int t,n,Is,rs,m;
4	scanf("%d",&t);
5	for(int i=0;i<t;i++){
6	Is=0;
7	rs=0;
8	scanf("%d",&n);

Sample Input 0

2
3
1 2 3
4
1 2 3 3

Sample Output 0

NO
YES

Explanation 0

For the first test case, no such index exists.
For the second test case, ***arr[0] + arr[1] = arr[3]***, therefore index **2** satisfies the given conditions.

Sample Input 1

3
5
1 1 4 1 1

Complete the code in the editor below. It should return a string, either YES if there is an element meeting the criterion or NO otherwise.

It has the following:

- `arr`: an array of integers

Input Format

The first line contains T , the number of test cases.

The next T pairs of lines each represent a test case.

- The first line contains n , the number of elements in the array `arr`.
- The second line contains n space-separated integers `arr[i]` where $0 \leq i < n$.

Constraints

- $1 \leq T \leq 10$
- $1 \leq n \leq 10^5$
- $1 \leq arr[i] \leq 2 \times 10^4$
- $0 \leq i \leq n$

Output Format

For each test case print YES if there exists an element in the array, such that the sum of the elements on its left is equal to the sum of the elements on its right; otherwise print NO.

```

33         co++;
34     }
35     int temp=ans[a];
36     ans[a]=ans[co];
37     ans[co]=temp;
38 }
39 for(int i=0;i<c1;i++){
40     printf("%d ",ans[i]);
41 }
42 return 0;
43 }
44

```

	Input	Expected	Got	
✓	10 203 204 205 206 207 208 203 204 205 206 13 203 204 204 205 206 207 205 208 203 206 205 206 204	204 205 206	204 205 206	✓

Passed all tests! ✓

Watson gives Sherlock an array of integers. His challenge is to find an element of the array such that the sum of all elements to the left is equal to the sum of all elements to the right. For instance, given the array **arr = [5, 6, 8, 11]**, **8** is between two subarrays that sum to **11**. If your starting array is **[1]**, that element satisfies the rule as left and right sum to **0**.

You will be given arrays of integers and must determine whether there is an element that meets the criterion.

204 is present in both arrays. Its frequency in **arr** is **2**, while its frequency in **brr** is **3**. Similarly, **205** and **206** occur twice in **arr**, but three times in **brr**. The rest of the numbers have the same frequencies in both lists.

Answer: (penalty regime: 0 %)

1	#include<stdio.h>	
2	int main(){	
3	int n,m,c,c1=0,co;	
4	scanf("%d",&n);	
5	int arr[n];	
6	for(int a=0;a<n;a++){	
7	scanf("%d",&arr[a]);	
8	}	
9	scanf("%d",&m);	
10	int brr[m],ans[m];	
11	for(int b=0;b<m;b++){	
12	scanf("%d",&brr[b]);	
13	}	
14	for(int j=0;j<m;j++){	
15	{	
16	c=0;	
17	for(int i=0;i<n;i++){	
18	if(arr[i]==brr[j]){	
19	c=1;	
20	arr[i]=-1;	
21	break;	
22	}	
23	}	
24	if(c==0){	
25	ans[c1]=brr[j];	
26	c1++;	
27	}	
28	}	
29	for(int a=0;a<c1;a++){	
30	co=0;	
31	for(int b=0;b<c1;b++){	
32	if(ans[b]<ans[a])	

- $1 \leq n, m \leq 2 \times 10^5$
- $n \leq m$
- $1 \leq brr[i] \leq 2 \times 10^4$
- $X_{max} - X_{min} < 101$

Output Format

Output the missing numbers in ascending order.

Sample Input

```
10
203 204 205 206 207 208 203 204 205 206
13
203 204 204 205 206 207 205 208 203 206 205 206 204
```

Sample Output

```
204 205 206
```

Explanation

Notes

- If a number occurs multiple times in the lists, you must ensure that the frequency of that number in both lists is the same. If that is not the case, then it is also a missing number.
- You have to print all the missing numbers in ascending order.
- Print each missing number once, even if it is missing multiple times.
- The difference between maximum and minimum number in the second list is less than or equal to **100**.

Complete the code in the editor below. It should return an array of missing numbers.

It has the following:

- `arr`: the array with missing numbers
- `brr`: the original array of numbers

Input Format

There will be four lines of input:

n - the size of the first list, ***arr***

The next line contains ***n*** space-separated integers ***arr[i]***

m - the size of the second list, ***brr***

The next line contains ***m*** space-separated integers ***brr[i]***

Constraints

```

14 1+(arr[a]+arr[b]==m){
15     printf("%d %d\n",a+1,b+1);
16     c=1;break;
17 }
18 } if(c==1) break;
19 }
20 }
21 return 0;
22 }

```

	Input	Expected	Got	
✓	2	1 4	1 4	✓
	4	1 2	1 2	
	5			
	1 4 5 3 2			
	4			
	4			
	2 2 4 3			

Passed all tests! ✓

Numeros the Artist had two lists that were permutations of one another. He was very proud. Unfortunately, while transporting them from one exhibition to another, some numbers were lost out of the first list. Can you find the missing numbers?

As an example, the array with some numbers missing, **arr** = [7, 2, 5, 3, 5, 3]. The original array of numbers **brr** = [7, 2, 5, 4, 6, 3, 5, 3]. The numbers missing are [4, 6].

2 2 4 3

Sample Output

1 4
1 2

Explanation

Sunny and Johnny make the following two trips to the parlor:

- 1. The first time, they pool together $m = 4$ dollars. Of the five flavors available that day, flavors **1** and **4** have a total cost of $1 + 3 = 4$.
- 2. The second time, they pool together $m = 4$ dollars. TOf the four flavors available that day, flavors **1** and **2** have a total cost of $2 + 2 = 4$.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main(){
3     int t,m,n,c=0;
4     scanf("%d",&t);
5     for(int i=0;i<t;i++){
6         c=0;
7         scanf("%d\n%d",&m,&n);
8         int arr[n];
9         for(int j=0;j<n;j++){
10             scanf("%d",&arr[j]);
11         }
12         for(int a=0;a<n-1;a++){
13             for(int b=a+1;b<n;b++){
```

Note: The index within the cost array represents the flavor of the ice cream purchased.

Constraints

- $1 \leq t \leq 50$
- $2 \leq m \leq 10^4$
- $2 \leq n \leq 10^4$
- $1 \leq cost[i] \leq 10^4, " i \in [1, n]$
- There will always be a unique solution.

Output Format

For each test case, print two space-separated integers denoting the indices of the two flavors purchased, in ascending order.

Sample Input

```
2
4
5
1 4 5 3 2
4
4
```

Sunny and Johnny like to pool their money and go to the ice cream parlor. Johnny never buys the same flavor that Sunny does. The only other rule they have is that they spend all of their money.

Given a list of prices for the flavors of ice cream, select the two that will cost all of the money they have.

For example, they have $m = 6$ to spend and there are flavors costing $cost = [1, 2, 3, 4, 5, 6]$. The two flavors costing **1** and **5** meet the criteria. Using **1**-based indexing, they are at indices **1** and **4**.

Function Description

Complete the code in the editor below. It should return an array containing the indices of the prices of the two flavors they buy.

It has the following:

- `m`: an integer denoting the amount of money they have to spend
- `cost`: an integer array denoting the cost of each flavor of ice cream

Input Format

The first line contains an integer, t , denoting the number of trips to the ice cream parlor. The next t sets of lines each describe a visit. Each trip is described as follows:

1. The integer m , the amount of money they have pooled.
2. The integer n , the number of flavors offered at the time.
3. n space-separated integers denoting the cost of each flavor: $cost[1], cost[2], \dots, cost[n]$.

--	--

	Input	Expected	Got	
✓	4	2	2	✓
	1	4	4	
	4			
	2			
	4			
	2			
	3			
	5			
✓	5	1	1	✓
	2	0	0	
	10	3	3	
	5	4	4	
	4			
	8			
	4			
	3			
	1			
	7			
	8			

Passed all tests! ✓

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int s1,s2,ans;
5     scanf("%d",&s1);
6     int ta[s1];
7     for(int i=0;i<s1;i++)
8     scanf("%d",&ta[i]);
9     scanf("%d",&s2);
10    int tb[s2];
11    for(int i=0;i<s2;i++)
12    scanf("%d",&tb[i]);
13    for(int j=0;j<s2;j++)
14    {
15        ans=0;
16        for(int i=0;i<s1;i++){
17            if(tb[j]>=ta[i])
18                ans++;
19        }printf("%d\n",ans);
20    }
21 }
```

	Input	Expected	Got	
✓	4	2	2	✓
	1	4	4	
	4			
	2			
	4			
	2			
	3			
	5			

4
3
1
7
8

Sample Output 1

1
0
3
4

Explanation 1

We are given, $n = 5$, $\text{nums} = [2, 10, 5, 4, 8]$, $m = 4$, and $\text{maxes} = [3, 1, 7, 8]$.

1. For $\text{maxes}[0] = 3$, we have 1 element in nums ($\text{nums}[0] = 2$) that is $\leq \text{maxes}[0]$.
2. For $\text{maxes}[1] = 1$, there are 0 elements in nums that are $\leq \text{maxes}[1]$.
3. For $\text{maxes}[2] = 7$, we have 3 elements in nums ($\text{nums}[0] = 2$, $\text{nums}[2] = 5$, and $\text{nums}[3] = 4$) that are $\leq \text{maxes}[2]$.
4. For $\text{maxes}[3] = 8$, we have 4 elements in nums ($\text{nums}[0] = 2$, $\text{nums}[2] = 5$, $\text{nums}[3] = 4$, and $\text{nums}[4] = 8$) that are $\leq \text{maxes}[3]$.

Thus, the function returns the array $[1, 0, 3, 4]$ as the answer.

Sample Output 0

2
4

Explanation 0

We are given $n = 4$, $\text{nums} = [1, 4, 2, 4]$, $m = 2$, and $\text{maxes} = [3, 5]$.

1. For $\text{maxes}[0] = 3$, we have 2 elements in nums ($\text{nums}[0] = 1$ and $\text{nums}[2] = 2$) that are $\leq \text{maxes}[0]$.
2. For $\text{maxes}[1] = 5$, we have 4 elements in nums ($\text{nums}[0] = 1$, $\text{nums}[1] = 4$, $\text{nums}[2] = 2$, and $\text{nums}[3] = 4$) that are $\leq \text{maxes}[1]$.

Thus, the function returns the array $[2, 4]$ as the answer.

Sample Case 1

Sample Input 1

5
2
10
5
4
8

Input Format For Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n , the number of elements in `nums`.

The next n lines each contain an integer describing `nums[j]` where $0 \leq j < n$.

The next line contains an integer m , the number of elements in `maxes`.

The next m lines each contain an integer describing `maxes[i]` where $0 \leq i < m$.

Sample Case 0

Sample Input 0

4

1

4

2

4

2

3

5

The number of goals achieved by two football teams in matches in a league is given in the form of two lists. Consider:

- Football team A, has played three matches, and has scored { 1 , 2 , 3 } goals in each match respectively.
- Football team B, has played two matches, and has scored { 2, 4 } goals in each match respectively.
- Your task is to compute, for each match of team B, the total number of matches of team A, where team A has scored less than or equal to the number of goals scored by team B in that match.
- In the above case:
 - For 2 goals scored by team B in its first match, team A has 2 matches with scores 1 and 2.
 - For 4 goals scored by team B in its second match, team A has 3 matches with scores 1, 2 and 3.

Hence, the answer: {2, 3}.

Complete the code in the editor below. The program must return an array of m positive integers, one for each `maxes[i]` representing the total number of elements `nums[j]` satisfying `nums[j] ≤ maxes[i]` where $0 \leq j < n$ and $0 \leq i < m$, in the given order.

It has the following:

`nums[nums[0],...nums[n-1]]`: first array of positive integers

`maxes[maxes[0],...maxes[m-1]]`: second array of positive integers

Constraints

- $2 \leq n, m \leq 105$
- $1 \leq \text{nums}[j] \leq 109$, where $0 \leq j < n$.
- $1 \leq \text{maxes}[i] \leq 109$, where $0 \leq i < m$.

```

10         if (x%2 != 0) c = c + 1;
11     }printf("%d\n",c);
12 }

```

	Input	Expected	Got	
✓	3	1	1	✓
	1	1	1	
	2	4	4	
	3			
✓	10	1296	1296	✓
	71	2500	2500	
	100	1849	1849	
	86	729	729	
	54	400	400	
	40	25	25	
	9	1521	1521	
	77	25	25	
	9	49	49	
	13	2401	2401	
	98			

Passed all tests! ✓

1
1
4

Explanation

Test Case 0: N = 1

Sam buys 1 chocolate on day 1, giving us a total of 1 chocolate. Thus, we print 1 on a new line.

Test Case 1: N = 2

Sam buys 1 chocolate on day 1 and 0 on day 2. This gives us a total of 1 chocolate. Thus, we print 1 on a new line.

Test Case 2: N = 3

Sam buys 1 chocolate on day 1, 0 on day 2, and 3 on day 3. This gives us a total of 4 chocolates. Thus, we print 4 on a new line.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main(){
3     int t;
4     scanf("%d",&t);
5     while(t--){
6         int n,c=0;
7         scanf("%d",&n);
8         for(int i=0;i<=n;i++){
9             if(i%2!=0) c=c+i;
```

The locked code in the editor handles reading the following input from stdin, assembling it into an array of integers (arr), and calling calculate(arr).

The first line of input contains an integer, T (the number of test cases). Each line i of the T subsequent lines describes the ith test case as an integer, Ni (the number of days).

Constraints

$$1 \leq T \leq 2 \times 10^5$$

$$1 \leq N \leq 2 \times 10^6$$

$$1 \leq x \leq N \leq Y$$

Output Format

For each test case, Ti in arr, your calculate method should print the total number of chocolates Sam purchased by day Ni on a new line.

Sample Input 0

```
3
1
2
3
```

Sample Output 0

```

20     if(flag) break;}
21     printf("%d\n",flag);
22     }
23 }
```

	Input	Expected	Got	
✓	1 3 1 3 5 4	1	1	✓
✓	1 3 1 3 5 99	0	0	✓

Passed all tests! ✓

Sam loves chocolates and starts buying them on the 1st day of the year. Each day of the year, x , is numbered from 1 to Y . On days when x is odd, Sam will buy x chocolates; on days when x is even, Sam will not purchase any chocolates.

Complete the code in the editor so that for each day N_i (where $1 \leq x \leq N \leq Y$) in array arr , the number of chocolates Sam purchased (during days 1 through N) is printed on a new line. This is a function-only challenge, so input is handled for you by the locked stub code in the editor.

Input Format

The program takes an array of integers as a parameter.

Input:

1
3 1 3 5
99

Output:

0

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main(){
3     int t;
4     scanf("%d",&t);
5     while(t--){
6         int n;
7         scanf("%d",&n);
8         int a[n];
9         for(int i=0;i<n;i++){
10             scanf("%d",&a[i]);
11         }
12
13         int k;
14         scanf("%d",&k);
15         int flag=0;
16         for(int i=0;i<n;i++){
17             for(int j=i+1;j<n;j++){
18                 if(a[i]-a[j]==k || a[j]-a[i]==k){flag=1;break;}
19             }
20         }
21         if(flag==1) printf("1\n");
22         else printf("0\n");
23     }
24 }
```

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[i] - A[j] = k$, $i \neq j$.

Input Format

1. First line is number of test cases T. Following T lines contain:
2. N, followed by N integers of the array
3. The non-negative integer k

Output format

Print 1 if such a pair exists and 0 if it doesn't.

Example

Input:

1

3 1 3 5

4

Output:

1

```

20         in=p3;
21     }
22     in=in-c;
23     p3=in;
24     for(i2=i;i2<n;i2++){
25         printf("%d",p3++);
26         if(i2!=n-1)printf ("0");
27     }printf("\n");
28     }
29 }
30 }

```

	Input	Expected	Got	
✓	3	Case #1	Case #1	✓
	3	10203010011012	10203010011012	
	4	**4050809	**4050809	
	5	****607	****607	
		Case #2	Case #2	
		1020304017018019020	1020304017018019020	
		**50607014015016	**50607014015016	
		****809012013	****809012013	
		*****10011	*****10011	
		Case #3	Case #3	
		102030405026027028029030	102030405026027028029030	
		**6070809022023024025	**6070809022023024025	
		****10011012019020021	****10011012019020021	
		*****13014017018	*****13014017018	
		*****15016	*****15016	

Passed all tests! ✓

**50607014015016

****809012013

*****10011

Case #3

102030405026027028029030

**6070809022023024025

****10011012019020021

*****13014017018

*****15016

Answer: (penalty regime: 0 %)

1	#include<stdio.h>	
2	int main(){	
3	int n,v,p3,c,in,i,i1,i2,t,ti;	
4	scanf("%d",&t);	
5	for(ti=0;ti<t;ti++){	
6	v=0;	
7	scanf("%d",&n);	
8	printf("Case #%d\n",ti+1);	
9	for(i=0;i<n;i++){	
10	c=0;	
11	if(i>0){	
12	for(i1=0;i1<i;i1++) printf("***");	
13	}	
14	for(i1=i;i1<n;i1++){	
15	if(i>0) c++;	
16	printf("%d0",++v);	
17	}	
18	if(i==0){	
19	p3=v+(v*(v-1))+1;	

First line contains T, the number of test cases

Each test case contains a single integer N

Output

First line print Case #i where i is the test case number

In the subsequent line, print the pattern

Test Case 1

3

3

4

5

Output

Case #1

10203010011012

**4050809

****607

Case #2

1020304017018019020

Decode the logic and print the Pattern that corresponds to given input.

If N= 3

then pattern will be :

10203010011012
**4050809
***607

If N= 4, then pattern will be:

1020304017018019020
**50607014015016
***809012013
****10011

Constraints

2 <= N <= 100

Input Format

```

4  int T,d,i,i1,i2,o,z;
5  char c,s;
6  scanf("%d",&T);
7  for(i=0;i<T;i++)
8  {
9      scanf("%d %c",&d,&s);
10     for(i1=0;i1<d;i1++)
11     {
12         z=(s=='W') ? 0:1;
13         o=(i1%2==z) ? 0:1;
14         for(i2=0;i2<d;i2++)
15         {
16             c=(i2%2==o)?'W':'B';
17             printf("%c",c);
18         }
19         printf("\n");
20     }
21 }
22 return 0;
23 }

```

	Input	Expected	Got	
✓	2	WB	WB	✓
	2 W	BW	BW	
	3 B	BWB	BWB	
		WBW	WBW	
		BWB	BWB	

Passed all tests! ✓

Output Format

Print the chessboard as per the given examples

Sample Input / Output

Input:

2

2 W

3 B

Output:

WB

BW

BWB

WBW

BWB

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
```

```

29 i1+=1;
30 printf("\n");
31 }
32 i=i+1;
33 }
34 }

```

	Input	Expected	Got	
✓	2	WBW	WBW	✓
	3	BWB	BWB	
	5	WBW	WBW	
		WBWBW	WBWBW	
		BWBWB	BWBWB	
		WBWBW	WBWBW	
		BWBWB	BWBWB	
		WBWBW	WBWBW	

Passed all tests! ✓

Let's print a chessboard!

Write a program that takes input:

The first line contains T, the number of test cases

Each test case contains an integer N and also the starting character of the chessboard

BWBWB

WBWBW

BWBWB

WBWBW

Answer: (penalty regime: 0 %)

1	#include<stdio.h>	
2	int main()	
3	{	
4	int T,d,i=0,i1,i2,o;	
5	char c;	
6	scanf("%d",&T);	
7	while(i<T)	
8	{	
9	scanf("%d",&d);	
10	i1=0;	
11	while(i1<d)	
12	{	
13	o=1;	
14	i2=0;	
15	if(i1%2==0)	
16	{	
17	o=0;	
18	}	
19	while(i2<d)	
20	{	
21	c='B';	
22	if(i2%2==0)	
23	{	
24	c='W';	
25	}	
26	printf("%c",c);	
27	i2++;	
28	}	

Write a program that prints a simple chessboard.

Input format:

The first line contains the number of inputs T.

The lines after that contain a different values for size of the chessboard

Output format:

Print a chessboard of dimensions size * size. Print a Print W for white spaces and B for black spaces.

Input:

2

3

5

Output:

WBW

BWB

WBW

WBWBW